



POO - CLASES

1. Continuando con el ejercicio “**Blade of Darnkness**” del tema anterior debes realizar lo siguiente:

- Las clases Monstruo y Jugador son muy similares. Sería interesante utilizar herencia para quitar parte del código repetido. Crea una nueva clase **Personaje** que sea padre de las otras dos y lleva a ella todas las propiedades y métodos comunes. Crea esa clase como **abstracta**. Los métodos subirNivel y reducirVida son iguales, por tanto, se llevan a la clase padre. El método golpear es diferente, así que lo llevamos a la clase padre Personaje, como abstracto y cada una de las clases hijas tendrá una implementación diferente. (Recordad que las propiedades en Personaje deben ser protected).
- Vamos a crear una nueva clase **MonstruoFinalNivel**, que herede de monstruo, añadirá como propiedades:
 - o **golpeEspecial**: serán los puntos de daño de un golpe especial que tiene este monstruo.
 - o **usosGolpeEspecial**: serán las veces que se ha usado el golpe especial. Se iniciará con tres, y decrementará cada vez que se use el golpe especial. Esta propiedad debe ser privada.
 - o Debéis sobrescribir el método golpear para que use el golpe especial.
- Se añade una clase nueva **Partida**. Sus atributos serán un jugador, un ArrayList de 10 (en principio) monstruos, 2 de los cuales serán MonstruoFinalNivel. Al constructor se le pasará un Jugador y, además, inicializará el ArrayList de Monstruos. Tendrá los siguientes métodos:
 - o **iniciarPartida()**: se llamará en el constructor. Le añadirá en un for 100 objetos Monstruos iguales. Equipará al Jugador con las armas que decidas.
 - o **turnoJugador()**: recorrerá el ArrayList de Monstruos, y el primero que encuentre le golpeará con sus armas. Si el enemigo tiene 0 o menos de vida se quitará del ArrayList de la partida. Si no quedan enemigos vivos en el ArrayList, el jugador gana la partida, y termina (return).
 - o **turnoEnemigos()**: el primer Monstruo vivo que haya en el ArrayList, golpea al jugador. Si el Jugador se queda sin vida, pierde la partida y el juego acaba (return).
- **TestJuego**. Crea un main, en él un objeto Partida, inicia la partida, y luego simula unos 30 turnos (for) alternativos de jugador y enemigo. Por último, muestra el resultado de quien gana, y cuántos enemigos destruye en total el jugador. Decide tú los valores de salud de cada uno, los puntos y daños del arma, de manera que esté ajustado el juego y el jugador pueda matar al menos 8 monstruos.



2. Granja inteligente

En una granja inteligente, los equipos están formados por diferentes máquinas agrícolas automatizadas que colaboran para optimizar las tareas del campo. Cada equipo se identifica por su nombre, su ubicación y el total de horas de operación de sus máquinas.

Clases y estructura del programa

Clase Granja

- Atributos:
 - nombre (String): nombre de la granja.
 - ubicacion (String): localización o comarca donde se ubica la granja.
 - totalHorasOperacion (double): representa la suma total de las horas trabajadas por todas las máquinas de la granja.
 - Maquinas (**List**<Maquina>) Una lista de máquinas que pertenecen a la granja.
- Los atributos deben ser **privados**.
- Métodos protegidos:
 - Constructor que inicializa atributos y crea la lista (ArrayList).
 - Getters y setters para cada atributo.
 - imprimirDatosGranja(): muestra el nombre, la ubicación y el total de horas de operación.
 - addMaquina(): permite añadir una máquina a la granja.
 - calcularTotalHoras(): actualiza el atributo totalHorasOperacion sumando las horas de todas las máquinas del equipo.
 - toString(): muestra los modelos de todas las máquinas registradas en la granja.
 - buscarMaquinaPorId(int id): busca y muestra los datos de una máquina concreta. Si no existe, devuelve null. Pista: *indexOf*

Interfaz Operable

```
public interface Operable {
    int APAGADA = 0;
    int ENCENDIDA = 1;
    int MANTENIMIENTO = 2;

    void encender();
    void apagar();
    void ponerEnMantenimiento();
}
```

Clase abstracta Maquina implementa Operable

Describe las características generales de toda máquina inteligente agrícola.

- Atributos (privados):
 - id (int): identificador único de la máquina.
 - modelo (String): nombre o modelo de la máquina.
 - estado (int)
 - horasUso (float): número de horas acumuladas de funcionamiento (inicialmente 0).
- Métodos:
 - Constructor que inicializa los atributos. El estado = APAGADA.
 - Getters y setters para cada atributo.
 - Método equals(): solo con el id
 - Método toString(): imprime los datos básicos de la máquina.
 - Método trabajar(float horas): incrementa el número de horas de funcionamiento si está ENCENDIDA.
 - Implementa los métodos del **interfaz**, básicamente cambia el estado al correspondiente.
 - Método abstracto imprimirTipo(), que devuelve un String con el tipo de máquina (según su subclase).



Subclases de Maquina:

Clase TractorAutonomo

Representa un tractor autónomo capaz de realizar labores de arado o transporte.

- Nuevos atributos (privados):
 - potenciaCV (float): potencia del motor en caballos de vapor.
 - consumoPorHora (float): consumo de combustible por hora (litros/hora).
- Métodos:
 - Constructor que llama al de la clase padre, más los nuevos atributos
 - Getters y setters nuevos
 - toString(): imprime los datos comunes y los propios.
 - imprimirTipo(): devuelve "Tractor Autónomo".

Clase DronRiego

Representa un dron usado para la pulverización o riego de precisión.

- Nuevos atributos (privados):
 - capacidadLitros (int): capacidad del tanque de agua o sustancia.
 - alcanceMetros (float): distancia máxima de vuelo.
- Métodos:
 - Constructor que llama al de la clase padre, más los nuevos atributos
 - Getters y setters nuevos
 - toString(): imprime los datos comunes y los propios.
 - imprimirTipo(): devuelve "Dron de Riego".

Clase CosechadoraAutonoma

Representa una máquina que cosecha de forma autónoma en la granja.

- Nuevos atributos (privados), por ejemplo:
 - anchoCorte (float): ancho de corte en metros.
 - rendimientoToneladasHora (float): cantidad de producto cosechado por hora.
- Métodos:
 - Constructor que llama al de la clase padre Maquina más los nuevos atributos.
 - Getters y setters de los nuevos atributos.
 - toString(): llama a super.imprimirDatos() y añade los datos propios.
 - imprimirTipo(): devuelve "Cosechadora Autónoma".

En una clase de prueba, en un método **main**:

- Crear una granja.
- Añadir máquinas de distintos tipos a la granja.
- Permitir al usuario interactuar con un **menú** que incluya las siguientes opciones:
 1. Imprimir datos de la granja
 2. Añadir una máquina (se piden sus datos)
 3. Añadir horas de operación máquina (se pide id maquina y horas trabajadas)
 4. Calcular el total de horas de operación
 5. Listar todas las máquinas
 6. Salir